We then build `generateRecordSet` to generate the `recordCount` number of records in each batch. Notice that in this notebook, we're using the `None` product type, so the records generated will only have four features: `CustomerID`, `TransactionTimestamp`, `Amount`, and `Label` (this will be important in the next chapter!).

```
# Generate a list of records
def generateRecordSet():
  Products = ["None"]
  Labels = [0,1]
  recordSet = []
  for Prod in Products:
    for Lab in Labels:
      recordSet.append(generateRecord(Prod, Lab))
  return reduce(pyspark.sql.dataframe.DataFrame.unionByName, recordSet)
```

Figure 2.24 – The generateRecordSet function creates a record for each product and each label. Each record contains nRows transactions

Finally, we write a function to generate a set of data, convert the data to a DataFrame, and write it out as one JSON file to a temporary path. Then, we move that file to the final volume destination.

```
def writeJsonFile(destination_path):
  recordDF = generateRecordSet()
  recordDF = recordDF.withColumn("Amount", expr("Amount / 100"))
  recordDF.coalesce(1).write.format("json").save(temp_path)

  tempJson = os.path.join(temp_path, dbutils.fs.ls(temp_path)[3][1])
  dbutils.fs.cp(tempJson, destination_path)
  dbutils.fs.rm(temp_path, True)
```

Figure 2.25 – The writeJsonFile function generates a set of records

The set contains amounts generated as integers, so we divide by 100 to turn the amounts into dollars and type float. The function writes out the JSON file to a `temp` directory and then moves the single file to the final directory.

With everything set up, create the dataset with the code provided. Feel free to increase the iterations to build a larger dataset. Then, move on to the next project!